



AWD COURSE

Platform Pembelajaran Pemrograman & Analisis Data

MODUL REFERENSI RESMI • BAB 2

Paket & Library Data Science Python

Panduan referensi komprehensif ekosistem data science: manajemen paket pip, komputasi array multidimensi NumPy, manipulasi DataFrame Pandas, visualisasi data Matplotlib & Seaborn, serta teknik impor dan ekspor data modern.

KATEGORI MATERI

Dasar Python & Analisis Data

FORMAT DOKUMEN

PDF Interaktif Siap Cetak

TINGKAT KESULITAN

Pemula s/d Menengah

AKSES ONLINE

course.awd.my.id

PEMATERI & PENYUSUN MATERI

I Putu Agus Wahyu Dupayana

Dipublikasikan secara resmi di AWD Course Platform

Edisi Pembaruan: **September 2026**

Bab 2: Ekosistem Paket & Sains Data Python

Awd Course – Buku Modul Praktik Komprehensif

Notebook kompilasi lengkap untuk Bab 2 yang mencakup Pip, NumPy, Pandas, Matplotlib, Seaborn, Impor/Ekspor Data CSV, dan Mini Proyek Analisis Inventaris.

Manajemen Paket & Library Menggunakan Pip

Pip (*Package Installer for Python*) adalah manajer paket standar resmi untuk bahasa pemrograman Python. Pip mempermudah proses pencarian, instalasi, pembaruan, dan penghapusan pustaka dari repositori publik **PyPI** (*Python Package Index*).

1. Memeriksa Versi Pip dan Daftar Paket

Gunakan perintah `!pip --version` dan `!pip list` di lingkungan notebook.

PYTHON 3

Contoh Kode Praktik

```
! pip --version
! pip list | head -n 10
```

OUTPUT EKSEKUSI:

```
pip 24.1.2 from /usr/local/lib/python3.13/dist-packages/pip (python 3.13)
```

Package	Version
absl-py	1.4.0
accelerate	1.14.0
access	1.1.10.post3
affine	3.0.1
aiofiles	25.1.0
aiohappyeyeballs	2.7.1
aiohttp	3.14.3
aiosignal	1.4.0

```
ERROR: Pipe to stdout was broken
Exception ignored on flushing sys.stdout:
BrokenPipeError: [Errno 32] Broken pipe
```

2. Menginstal dan Memeriksa Informasi Paket

Anda dapat menginstal pustaka seperti `requests` menggunakan perintah `pip install`.

```
! pip install requests
```

OUTPUT EKSEKUSI:

```
Requirement already satisfied: requests in /usr/local/lib/python3.13/dist-packages (2.32.4)  
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages  
(from requests) (3.4.9)  
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from  
requests) (3.19)  
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from  
requests) (2.5.0)  
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from  
requests) (2026.7.22)
```

3. Menggunakan Paket yang Telah Terinstal

```
import requests  
  
# Mengambil status API publik  
response = requests.get('https://api.github.com')  
print("Status Code:", response.status_code)  
print("Server Headers:", response.headers.get('server'))
```

OUTPUT EKSEKUSI:

```
Status Code: 200  
Server Headers: github.com
```

4. Memeriksa Informasi Detail Paket (pip show)

Untuk melihat metadata lengkap paket seperti versi rilis, lokasi instalasi (*site-packages*), lisensi, dan dependensi pustaka, gunakan perintah `!pip show`.

```
! pip show requests
```

OUTPUT EKSEKUSI:

```
Name: requests
Version: 2.32.4
Summary: Python HTTP for Humans.
Home-page: https://requests.readthedocs.io
Author: Kenneth Reitz
Author-email: me@kennethreitz.org
License: Apache-2.0
Location: /usr/local/lib/python3.13/dist-packages
Requires: certifi, charset_normalizer, idna, urllib3
Required-by: bigframes, CacheControl, community, datasets, diffusers, earthengine-api, fastai, folium, gcsfs, gdown, geemap, geocoder, google-adk, google-api-core, google-cloud-bigquery, google-cloud-storage, google-colab, google-genai, googledrivedownloader, jupyter_kernel_gateway, kaggle, kagglehub, kagglesdk, keyrings.google-artifactregistry-auth, langsmith, libpysal, moviepy, music21, opentelemetry-resourcedetector-gcp, pandas-datareader, panel, pooch, pyiceberg, pysal, quantecon, requests-oauthlib, requests-toolbelt, spacy, spanner-graph-notebook, Sphinx, tensorflow, tensorflow-datasets, tiktoken, torchdata, tsfresh, tweepy, wandb, yfinance
```

5. Membaca Dokumentasi dan Parameter Fungsi

Fungsi `help()` bawaan Python: Masukkan nama pustaka, modul, atau fungsi spesifik ke dalam `help()` untuk menampilkan docstring resmi, deskripsi fungsi, daftar argumen/parameter, nilai default, serta tipe kembalian (return type).

```
help(requests)
```

OUTPUT EKSEKUSI:

Help on package requests:

NAME

requests

DESCRIPTION

Requests HTTP Library

~~~~~

Requests is an HTTP library, written in Python, for human beings.  
Basic GET usage:

```
>>> import requests
>>> r = requests.get('https://www.python.org')
>>> r.status_code
200
>>> b'Python is a programming language' in r.content
True
```

... or POST:

```
>>> payload = dict(key1='value1', key2='value2')
>>> r = requests.post('https://httpbin.org/post', data=payload)
>>> print(r.text)
{
  ...
  "form": {
    "key1": "value1",
    "key2": "value2"
  },
  ...
}
```

The other HTTP methods are supported - see `requests.api`. Full documentation is at <<https://requests.readthedocs.io>>.

:copyright: (c) 2017 by Kenneth Reitz.

:license: Apache 2.0, see LICENSE for more details.

**PACKAGE CONTENTS**

```
__version__
_internal_utils
adapters
api
auth
certs
compat
cookies
exceptions
```

```

help
hooks
models
packages
sessions
status_codes
structures
utils

FUNCTIONS
    check_compatibility(
        urllib3_version,
        chardet_version,
        charset_normalizer_version
    )

DATA
    __author_email__ = 'me@kennethreitz.org'
    __build__ = 143876
    __cake__ = '🌟🍰🌟'
    __copyright__ = 'Copyright Kenneth Reitz'
    __description__ = 'Python HTTP for Humans.'
    __license__ = 'Apache-2.0'
    __title__ = 'requests'
    __url__ = 'https://requests.readthedocs.io'
    chardet_version = '5.2.0'
    charset_normalizer_version = '3.4.9'
    codes = <lookup 'status_codes'>

VERSION
    2.32.4

AUTHOR
    Kenneth Reitz

FILE
    /usr/local/lib/python3.13/dist-packages/requests/__init__.py

```

## 6. Melihat Daftar Semua Fungsi dan Atribut yang Tersedia

Fungsi `dir()`: Mengembalikan seluruh daftar metode, kelas, konstanta, dan fungsi yang ada di dalam suatu modul atau objek dalam bentuk list of strings.

```
print(dir(requests))
```

**OUTPUT EKSEKUSI:**

```
['ConnectTimeout', 'ConnectionError', 'DependencyWarning', 'FileModeWarning', 'HTTPError',  
'JSONDecodeError', 'NullHandler', 'PreparedRequest', 'ReadTimeout', 'Request', 'RequestException',  
'RequestsDependencyWarning', 'Response', 'Session', 'Timeout', 'TooManyRedirects', 'URLRequired',  
'__author__', '__author_email__', '__build__', '__builtins__', '__cached__', '__cake__',  
'__copyright__', '__description__', '__doc__', '__file__', '__license__', '__loader__', '__name__',  
'__package__', '__path__', '__spec__', '__title__', '__url__', '__version__',  
'_check_cryptography', '_internal_utils', 'adapters', 'api', 'auth', 'certs', 'chardet_version',  
'charset_normalizer_version', 'check_compatibility', 'codes', 'compat', 'cookies', 'delete',  
'exceptions', 'get', 'head', 'hooks', 'logging', 'models', 'options', 'packages', 'patch', 'post',  
'put', 'request', 'session', 'sessions', 'ssl', 'status_codes', 'structures', 'urllib3', 'utils',  
'warnings']
```

## 7. Mengetahui Parameter Fungsi Secara Ringkas (inspect)

Modul inspect: Berguna untuk melihat struktur parameter secara persis tanpa membaca teks panjang:

```
import inspect  
print(inspect.signature(requests.get))
```

**OUTPUT EKSEKUSI:**

```
(url, params=None, **kwargs)
```

## 8. Menghapus (Uninstall) Paket dengan Pip

Untuk menghapus paket atau pustaka yang sudah tidak diperlukan dari sistem, gunakan perintah `pip uninstall`.

Pada lingkungan Jupyter Notebook atau skrip otomatisasi, tambahkan parameter `-y` (*Yes to confirmation*) agar proses pencopotan paket berjalan langsung tanpa menunggu konfirmasi manual di terminal.

```
# Menghapus pustaka requests  
! pip uninstall requests -y
```

**OUTPUT EKSEKUSI:**

```
Found existing installation: requests 2.32.4  
Uninstalling requests-2.32.4:  
  Successfully uninstalled requests-2.32.4
```

## Memasang Kembali Pustaka untuk Praktik Selanjutnya

Setelah berhasil menguji perintah `uninstall`, pasang kembali paket `requests` agar siap digunakan pada materi-materi berikutnya.

PYTHON 3

Contoh Kode Praktik

```
# Memasang kembali pustaka requests
! pip install requests
```

### OUTPUT EKSEKUSI:

```
Collecting requests
  Downloading requests-2.34.2-py3-none-any.whl.metadata (4.8 kB)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests) (3.4.9)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from requests) (3.19)
Requirement already satisfied: urllib3<3,>=1.26 in /usr/local/lib/python3.13/dist-packages (from requests) (2.5.0)
Requirement already satisfied: certifi>=2023.5.7 in /usr/local/lib/python3.13/dist-packages (from requests) (2026.7.22)
Downloading requests-2.34.2-py3-none-any.whl (73 kB)
[2K  [90m-----[0m [32m73.1/73.1 kB[0m [31m3.1 MB/s[0m
eta [36m0:00:00[0m
[?25hInstalling collected packages: requests
[31mERROR: pip's dependency resolver does not currently take into account all the packages that
are installed. This behaviour is the source of the following dependency conflicts.
google-colab 1.0.0 requires requests==2.32.4, but you have requests 2.34.2 which is
incompatible.[0m[31m
[0mSuccessfully installed requests-2.34.2
```

## Pengenalan & Komputasi Array NumPy

**NumPy** (*Numerical Python*) adalah pustaka dasar untuk komputasi ilmiah dalam Python. NumPy menyediakan struktur data **ndarray** (*N-dimensional array*) yang jauh lebih cepat dan hemat memori dibandingkan list bawaan Python.

### 1. Instalasi Pustaka NumPy

Sebelum menggunakan NumPy di lingkungan lokal atau Google Colab, pastikan pustaka sudah terpasang menggunakan perintah `pip install`:

PYTHON 3

Contoh Kode Praktik

```
! pip install numpy
```

### OUTPUT EKSEKUSI:

```
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (2.1.3)
```

## 2. Inisiasi dan Membuat Array NumPy

PYTHON 3

Contoh Kode Praktik

```
import numpy as np

# Membuat Array 1 Dimensi (Vektor)
arr_1d = np.array([10, 20, 30, 40, 50])
print("Array 1D:", arr_1d)
print("Bentuk (shape):", arr_1d.shape)
print("Tipe data (dtype):", arr_1d.dtype)

# Membuat Array 2 Dimensi (Matriks)
matrix_2d = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])
print("\nMatriks 2D:\n", matrix_2d)
print("Dimensi (ndim):", matrix_2d.ndim)
print("Ukuran (size):", matrix_2d.size)
```

### OUTPUT EKSEKUSI:

```
Array 1D: [10 20 30 40 50]
Bentuk (shape): (5,)
Tipe data (dtype): int64

Matriks 2D:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
Dimensi (ndim): 2
Ukuran (size): 9
```

## 3. Fungsi Pembuat Array Otomatis

NumPy menyediakan fungsi instan untuk membuat array bernilai nol, satu, deret, atau nilai acak.

```
# Array berisi nol (zeros)
zeros = np.zeros((2, 4))

# Array berisi deret angka urut (arange)
deret = np.arange(0, 20, 2) # start=0, stop=20, step=2

# Array linear space (linspace)
lin = np.linspace(0, 1, 5) # 5 titik berjarak sama dari 0 s.d. 1

print("Zeros (2x4):\n", zeros)
print("Arange (kelipatan 2):", deret)
print("Linspace (0 s.d. 1):", lin)
```

**OUTPUT EKSEKUSI:**

```
Zeros (2x4):
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]]
Arange (kelipatan 2): [ 0  2  4  6  8 10 12 14 16 18]
Linspace (0 s.d. 1): [0.   0.25 0.5  0.75 1.   ]
```

## 4. Operasi Matematika Vektorisasi (Vectorization)

Salah satu keunggulan terbesar NumPy adalah operasi aritmatika dieksekusi secara instan per elemen (*element-wise*) tanpa perlu menuliskan perulangan `for`.

```
a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])

print("Penjumlahan (a + b) =", a + b)
print("Perkalian (a * 3)   =", a * 3)
print("Kuadrat (a ** 2)    =", a ** 2)
print("Sinus np.sin(a)     =", np.round(np.sin(a), 4))
```

**OUTPUT EKSEKUSI:**

```
Penjumlahan (a + b) = [11 22 33 44]
Perkalian (a * 3)   = [ 3  6  9 12]
Kuadrat (a ** 2)    = [ 1  4  9 16]
Sinus np.sin(a)     = [ 0.8415  0.9093  0.1411 -0.7568]
```

## 5. Fungsi Agregasi Statistik NumPy

PYTHON 3

Contoh Kode Praktik

```
data_sampel = np.array([45, 82, 67, 90, 54, 78, 88, 95, 72, 84])

print("Rata-rata (Mean)      :", np.mean(data_sampel))
print("Median (Nilai Tengah) :", np.median(data_sampel))
print("Standar Deviasi (Std) :", np.std(data_sampel))
print("Varians (Variance)    :", np.var(data_sampel))
print("Nilai Minimum         :", np.min(data_sampel))
print("Nilai Maksimum        :", np.max(data_sampel))
```

### OUTPUT EKSEKUSI:

```
Rata-rata (Mean)      : 75.5
Median (Nilai Tengah) : 80.0
Standar Deviasi (Std) : 15.311760186209813
Varians (Variance)    : 234.45
Nilai Minimum         : 45
Nilai Maksimum        : 95
```

## Pengenalan Struktur Data Pandas (Series & DataFrame)

**Pandas** adalah pustaka paling populer dalam Python untuk manipulasi, pembersihan, dan analisis data terstruktur.

### 1. Instalasi Pustaka Pandas

Sebelum memulai analisis data tabular, pastikan pustaka Pandas telah terpasang pada lingkungan Python Anda:

PYTHON 3

Contoh Kode Praktik

```
! pip install pandas
```

### OUTPUT EKSEKUSI:

```
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (2.2.3)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.13/dist-packages (from
pandas) (2.1.3)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.13/dist-packages
(from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages (from
pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages (from
pandas) (2026.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-
dateutil>=2.8.2->pandas) (1.17.0)
```

## 2. Pandas Series (Struktur 1 Dimensi Berlabel)

Series adalah array satu dimensi yang dilengkapi dengan label indeks.

PYTHON 3

Contoh Kode Praktik

```
import pandas as pd

# Membuat Series dengan indeks kustom
populasi_juta = pd.Series([10.56, 2.93, 2.51, 1.83], index=['Jakarta', 'Surabaya', 'Bekasi', 'Bandung'])
populasi_juta.name = "Populasi (Juta Jiwa)"

print(populasi_juta)
print("\nPopulasi Surabaya:", populasi_juta['Surabaya'], "juta")
```

### OUTPUT EKSEKUSI:

```
Jakarta      10.56
Surabaya      2.93
Bekasi        2.51
Bandung       1.83
Name: Populasi (Juta Jiwa), dtype: float64

Populasi Surabaya: 2.93 juta
```

## 3. Pandas DataFrame (Struktur Tabel 2 Dimensi)

DataFrame adalah representasi tabel dua dimensi yang terdiri dari baris (*rows*) dan kolom (*columns*).

PYTHON 3

Contoh Kode Praktik

```
# Membuat DataFrame dari Dictionary
data_karyawan = {
    'Nama': ['Andi', 'Budi', 'Citra', 'Dewi', 'Eko'],
    'Divisi': ['Engineering', 'Data', 'Data', 'Design', 'Engineering'],
    'Gaji_Juta': [15.5, 18.0, 16.5, 12.0, 14.0],
    'Pengalaman_Tahun': [3, 5, 4, 2, 3]
}

df = pd.DataFrame(data_karyawan)
print("DataFrame Karyawan:\n", df)
```

### OUTPUT EKSEKUSI:

```
DataFrame Karyawan:
   Nama  Divisi  Gaji_Juta  Pengalaman_Tahun
0  Andi  Engineering     15.5                3
1  Budi    Data     18.0                5
2  Citra    Data     16.5                4
3  Dewi   Design     12.0                2
4  Eko   Engineering     14.0                3
```

## 4. Inspeksi Cepat & Ringkasan DataFrame

PYTHON 3

Contoh Kode Praktik

```
# Mengambil kolom tertentu
print("Daftar Gaji Karyawan:\n", df['Gaji_Juta'])

# Ringkasan statistik cepat
print("\nStatistik Gaji:\n", df['Gaji_Juta'].describe())

# Rata-rata gaji per divisi
print("\nRata-rata Gaji per Divisi:\n", df.groupby('Divisi')['Gaji_Juta'].mean())
```

### OUTPUT EKSEKUSI:

Daftar Gaji Karyawan:

```
0    15.5
1    18.0
2    16.5
3    12.0
4    14.0
```

Name: Gaji\_Juta, dtype: float64

Statistik Gaji:

```
count    5.000000
mean     15.200000
std       2.307596
min      12.000000
25%      14.000000
50%      15.500000
75%      16.500000
max      18.000000
```

Name: Gaji\_Juta, dtype: float64

Rata-rata Gaji per Divisi:

```
Divisi
Data      17.25
Design    12.00
Engineering 14.75
```

Name: Gaji\_Juta, dtype: float64

## Visualisasi Data Dasar dengan Matplotlib

**Matplotlib** adalah pustaka visualisasi data tingkat dasar yang memberikan kontrol penuh atas setiap elemen grafik (garis, sumbu, label, warna, dan legenda).

### 1. Instalasi Pustaka Matplotlib

Sebelum membuat diagram dan visualisasi grafik data, pastikan pustaka Matplotlib telah terpasang:

```
! pip install matplotlib
```

**OUTPUT EKSEKUSI:**

```
Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/dist-packages (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dist-packages (from
matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages (from
matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/dist-packages (from
matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/dist-packages (from
matplotlib) (1.5.0)
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.13/dist-packages (from
matplotlib) (2.1.3)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from
matplotlib) (26.3)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.13/dist-packages (from
matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/dist-packages (from
matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.13/dist-packages
(from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-
dateutil>=2.7->matplotlib) (1.17.0)
```

## 2. Inisiasi & Membuat Line Plot (Tren Waktu)

Line Plot sangat ideal untuk menampilkan tren data deret waktu (*time series*).

```
import matplotlib.pyplot as plt
import pandas as pd

# Data tren penjualan bulanan
bulan = ['Jan', 'Feb', 'Mar', 'Apr', 'Mei', 'Jun', 'Jul', 'Agu']
penjualan_2025 = [120, 135, 148, 160, 175, 190, 210, 230]
penjualan_2026 = [140, 150, 170, 185, 205, 220, 245, 270]

plt.figure(figsize=(10, 5))
plt.plot(bulan, penjualan_2025, marker='o', linestyle='--', color='#2563eb', label='Tahun 2025')
plt.plot(bulan, penjualan_2026, marker='s', linestyle='--', color='#059669', label='Tahun 2026')

plt.title('Perbandingan Tren Penjualan Bulanan (2025 vs 2026)', fontsize=14, fontweight='bold')
plt.xlabel('Bulan', fontsize=11)
plt.ylabel('Total Penjualan (Unit)', fontsize=11)
plt.legend()
plt.grid(True, linestyle=':', alpha=0.6)
plt.tight_layout()
plt.show()
```

**OUTPUT EKSEKUSI:**

<Figure size 1000x500 with 1 Axes>

### 3. Bar Plot (Perbandingan Antar Kategori)

Bar Plot digunakan untuk membandingkan besaran nilai diskrit antar kategori.

```
kategori_kursus = ['Python', 'SQL', 'Data Science', 'Web Dev', 'Cloud']
jumlah_peserta = [450, 380, 520, 290, 310]

plt.figure(figsize=(9, 5))
bars = plt.bar(kategori_kursus, jumlah_peserta, color='#3b82f6', edgecolor='#1e40af')

# Menambahkan label angka di atas setiap batang
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 8, f"{int(yval)}", ha='center',
             fontweight='bold')

plt.title('Jumlah Peserta Berdasarkan Kategori Kursus', fontsize=13, fontweight='bold')
plt.xlabel('Kategori Kursus')
plt.ylabel('Jumlah Siswa Terdaftar')
plt.ylim(0, 600)
plt.grid(axis='y', linestyle=':', alpha=0.7)
plt.tight_layout()
plt.show()
```

**OUTPUT EKSEKUSI:**

<Figure size 900x500 with 1 Axes>

## 4. Histogram & Scatter Plot

- **Histogram:** Melihat sebaran frekuensi data numerik kontinu.
- **Scatter Plot:** Memeriksa korelasi / hubungan antara dua variabel kuantitatif.

```
import numpy as np

# 1. Histogram Nilai Ujian
np.random.seed(42)
nilai_mahasiswa = np.random.normal(loc=78, scale=8, size=200)

plt.figure(figsize=(10, 4))
plt.hist(nilai_mahasiswa, bins=15, color='#8b5cf6', edgecolor='white')
plt.title('Distribusi Sebaran Nilai Mahasiswa', fontsize=13, fontweight='bold')
plt.xlabel('Rentang Nilai')
plt.ylabel('Frekuensi')
plt.grid(axis='y', linestyle=':', alpha=0.6)
plt.show()

# 2. Scatter Plot: Jam Belajar vs Nilai Ujian
jam_belajar = np.random.uniform(2, 10, size=50)
nilai_ujian = 50 + (jam_belajar * 4.5) + np.random.normal(0, 3, size=50)

plt.figure(figsize=(9, 5))
plt.scatter(jam_belajar, nilai_ujian, color='#ef4444', alpha=0.8, edgecolors='black')
plt.title('Hubungan Waktu Belajar terhadap Nilai Ujian', fontsize=13, fontweight='bold')
plt.xlabel('Waktu Belajar (Jam / Minggu)')
plt.ylabel('Nilai Ujian Akhir')
plt.grid(True, linestyle=':', alpha=0.6)
plt.show()
```

OUTPUT EKSEKUSI:

<Figure size 1000x400 with 1 Axes>

OUTPUT EKSEKUSI:

<Figure size 900x500 with 1 Axes>

## Visualisasi Data Statistik dengan Seaborn

**Seaborn** adalah pustaka visualisasi tingkat tinggi yang dibangun di atas Matplotlib. Seaborn dirancang untuk menghasilkan grafik statistik yang indah dan informatif secara otomatis.

### 1. Instalasi Pustaka Seaborn

Sebelum menyusun grafik visualisasi statistik tingkat tinggi, pastikan pustaka Seaborn telah terpasang:

```
! pip install seaborn
```

**OUTPUT EKSEKUSI:**

```
Requirement already satisfied: seaborn in /usr/local/lib/python3.13/dist-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in /usr/local/lib/python3.13/dist-packages
(from seaborn) (2.1.3)
Requirement already satisfied: pandas>=1.2 in /usr/local/lib/python3.13/dist-packages (from
seaborn) (2.2.3)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /usr/local/lib/python3.13/dist-packages
(from seaborn) (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dist-packages (from
matplotlib!=3.6.1,>=3.4->seaborn) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages (from
matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/dist-packages (from
matplotlib!=3.6.1,>=3.4->seaborn) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/dist-packages (from
matplotlib!=3.6.1,>=3.4->seaborn) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from
matplotlib!=3.6.1,>=3.4->seaborn) (26.3)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.13/dist-packages (from
matplotlib!=3.6.1,>=3.4->seaborn) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/dist-packages (from
matplotlib!=3.6.1,>=3.4->seaborn) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.13/dist-packages
(from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages (from
pandas>=1.2->seaborn) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages (from
pandas>=1.2->seaborn) (2026.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-
dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)
```

## 2. Inisiasi & Menyiapkan Dataset Sampel

PYTHON 3

Contoh Kode Praktik

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Mengatur tema visual modern bawaan seaborn
sns.set_theme(style="whitegrid")

# Membuat dataset transaksi e-commerce
data_transaksi = pd.DataFrame({
    'Kategori': ['Elektronik', 'Pakaian', 'Elektronik', 'Buku', 'Pakaian', 'Elektronik',
    'Buku', 'Pakaian', 'Elektronik', 'Buku'] * 10,
    'Rating': [4.5, 4.2, 4.8, 3.9, 4.1, 4.6, 4.0, 3.8, 4.9, 4.3] * 10,
    'Harga_Ribu': [1200, 250, 1800, 95, 320, 2100, 120, 180, 1500, 85] * 10,
    'Metode_Bayar': ['QRIS', 'Transfer', 'Kartu Kredit', 'QRIS', 'QRIS', 'Kartu Kredit',
    'Transfer', 'QRIS', 'Kartu Kredit', 'Transfer'] * 10
})

data_transaksi.head()
```

OUTPUT EKSEKUSI:

|   | Kategori   | Rating | Harga_Ribu | Metode_Bayar |
|---|------------|--------|------------|--------------|
| 0 | Elektronik | 4.5    | 1200       | QRIS         |
| 1 | Pakaian    | 4.2    | 250        | Transfer     |
| 2 | Elektronik | 4.8    | 1800       | Kartu Kredit |
| 3 | Buku       | 3.9    | 95         | QRIS         |
| 4 | Pakaian    | 4.1    | 320        | QRIS         |

## 3. Barplot dengan Kategorisasi Warna ( `hue` )

PYTHON 3

Contoh Kode Praktik

```
plt.figure(figsize=(10, 5))
sns.barplot(data=data_transaksi, x='Kategori', y='Harga_Ribu', hue='Metode_Bayar',
palette='Set2')
plt.title('Rata-rata Nilai Transaksi per Kategori & Metode Pembayaran', fontsize=13,
fontweight='bold')
plt.ylabel('Rata-rata Harga (Ribuan Rp)')
plt.show()
```

OUTPUT EKSEKUSI:

<Figure size 1000x500 with 1 Axes>

## 4. Histogram Distribusi dengan Kurva KDE ( `sns.histplot` )

KDE (*Kernel Density Estimate*) memperlihatkan perkiraan kurva kepadatan probabilitas kontinu.

```
plt.figure(figsize=(9, 5))
sns.histplot(data=data_transaksi, x='Rating', kde=True, color='#0284c7', bins=10)
plt.title('Sebaran Distribusi Rating Produk dengan Kurva KDE', fontsize=13,
fontweight='bold')
plt.xlabel('Rating Produk (Skala 1 - 5)')
plt.show()
```

**OUTPUT EKSEKUSI:**

<Figure size 900x500 with 1 Axes>

## 5. Scatter Plot & Boxplot Statistik Multi-Dimensi

```
plt.figure(figsize=(9, 5))
sns.boxplot(data=data_transaksi, x='Kategori', y='Harga_Ribu', palette='pastel')
plt.title('Distribusi Variasi Harga per Kategori Produk (Boxplot)', fontsize=13,
fontweight='bold')
plt.ylabel('Harga Produk (Ribu Rp)')
plt.show()
```

**OUTPUT EKSEKUSI:**

/tmp/ipykernel\_977/3631864634.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data=data_transaksi, x='Kategori', y='Harga_Ribu', palette='pastel')
```

**OUTPUT EKSEKUSI:**

<Figure size 900x500 with 1 Axes>

## Teknik Impor & Ekspor Data (CSV, URL, Git, Google Drive, Excel, JSON)

Dalam proyek sains data di dunia industri, dataset jarang dibuat secara manual menggunakan kode, melainkan dimuat dari berbagai sumber penyimpanan: berkas lokal (CSV/Excel), tautan web langsung (URL), repositori Git, Google Drive (Google Colab), maupun API berbasis JSON. Modul ini membahas seluruh metode impor dan ekspor data paling esensial.

### 1. Membaca & Menulis Berkas CSV Lokal ( `pd.read_csv` & `df.to_csv` )

CSV (*Comma Separated Values*) adalah format pertukaran data tabular yang paling umum digunakan dalam analisis data.

```
import pandas as pd

# 1. Membuat data sampel produk
df_produk = pd.DataFrame({
    'Kode': ['A01', 'A02', 'A03', 'A04'],
    'Produk': ['Laptop Pro', 'Mouse Wireless', 'Keyboard Mekanikal', 'Monitor 4K'],
    'Harga': [14500000, 250000, 850000, 4200000],
    'Stok': [12, 45, 28, 15]
})

# 2. Ekspor ke berkas CSV lokal (gunakan index=False agar nomor indeks tidak ikut tersimpan)
df_produk.to_csv('daftar_produk.csv', index=False)
print("Berkas 'daftar_produk.csv' berhasil disimpan!")

# 3. Membaca kembali berkas CSV
df_baca = pd.read_csv('daftar_produk.csv')
print("\nHasil Baca Berkas CSV:\n", df_baca)
```

**OUTPUT EKSEKUSI:**

Berkas 'daftar\_produk.csv' berhasil disimpan!

Hasil Baca Berkas CSV:

|   | Kode | Produk             | Harga    | Stok |
|---|------|--------------------|----------|------|
| 0 | A01  | Laptop Pro         | 14500000 | 12   |
| 1 | A02  | Mouse Wireless     | 250000   | 45   |
| 2 | A03  | Keyboard Mekanikal | 850000   | 28   |
| 3 | A04  | Monitor 4K         | 4200000  | 15   |

**Parameter Penting pada `pd.read_csv` :**

- **sep** atau **delimiter** : Menentukan pemisah kolom jika menggunakan titik koma ( `;` ) atau tab ( `\t` ).
- **usecols** : Hanya membaca kolom-kolom tertentu untuk menghemat penggunaan RAM.
- **nrows** : Membatasi jumlah baris yang dibaca (sangat berguna untuk mengintip sampel awal berkas raksasa berukuran gigabyte).
- **encoding** : Mengatur standar pengkodean karakter, misalnya `'utf-8'` atau `'latin1'` .

```
# Contoh membaca hanya kolom 'Produk' dan 'Harga' serta membatasi 2 baris pertama
df_spesifik = pd.read_csv('daftar_produk.csv', usecols=['Produk', 'Harga'], nrows=2)
print("Baca Spesifik (Kolom Tertentu & Batas Baris):\n", df_spesifik)
```

**OUTPUT EKSEKUSI:**

Baca Spesifik (Kolom Tertentu & Batas Baris):

|   | Produk         | Harga    |
|---|----------------|----------|
| 0 | Laptop Pro     | 14500000 |
| 1 | Mouse Wireless | 250000   |

## 2. Membaca Dataset Langsung dari URL Web (Online Dataset)

Fungsi `pd.read_csv` di Pandas dapat langsung menerima alamat tautan URL HTTP/HTTPS (seperti URL *raw* file di GitHub atau portal Open Data), sehingga Anda tidak perlu mengunduh berkas secara manual terlebih dahulu ke komputer.

PYTHON 3

Contoh Kode Praktik

```
# Membaca dataset publik langsung dari GitHub Raw URL
url_dataset = 'https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv'
df_online = pd.read_csv(url_dataset)

print(f"Dataset berhasil dimuat dari URL! Dimensi: {df_online.shape[0]} baris,
{df_online.shape[1]} kolom.")
print("\n5 Baris Pertama:\n", df_online.head())
```

### OUTPUT EKSEKUSI:

Dataset berhasil dimuat dari URL! Dimensi: 150 baris, 5 kolom.

5 Baris Pertama:

|   | sepal_length | sepal_width | petal_length | petal_width | species |
|---|--------------|-------------|--------------|-------------|---------|
| 0 | 5.1          | 3.5         | 1.4          | 0.2         | setosa  |
| 1 | 4.9          | 3.0         | 1.4          | 0.2         | setosa  |
| 2 | 4.7          | 3.2         | 1.3          | 0.2         | setosa  |
| 3 | 4.6          | 3.1         | 1.5          | 0.2         | setosa  |
| 4 | 5.0          | 3.6         | 1.4          | 0.2         | setosa  |

## 3. Mengunduh & Mengimpor Data Menggunakan Git ( `git clone` )

Jika proyek riset atau tim Anda menyimpan kumpulan dataset, gambar, dan skrip pada repositori GitHub/GitLab, Anda dapat menarik seluruh repositori ke lingkungan kerja menggunakan perintah `!git clone` .

PYTHON 3

Contoh Kode Praktik

```
# Contoh perintah mengunduh repositori dataset dari GitHub di notebook/terminal:
# ! git clone https://github.com/nama-pengguna/nama-repositori-data.git

# Setelah repositori selesai di-clone, berkas dapat dibaca melalui jalur foldernya:
# df_repo = pd.read_csv('nama-repositori-data/data_penjualan.csv')
print("Perintah Git Clone: !git clone <URL_REPOSITORI_GIT>")
print("Akses berkas setelah di-clone: pd.read_csv('folder_hasil_clone/file.csv')")
```

### OUTPUT EKSEKUSI:

Perintah Git Clone: !git clone <URL\_REPOSITORI\_GIT>

Akses berkas setelah di-clone: pd.read\_csv('folder\_hasil\_clone/file.csv')

## 4. Menghubungkan & Mengakses Data dari Google Drive (Google Colab)

## Workflow)

Pada lingkungan **Google Colab**, penyimpanan lokal bersifat sementara (*ephemeral*) dan akan hilang saat runtime di-restart. Untuk menyimpan atau memuat dataset berukuran besar secara persisten, praktisi data mengaitkan (*mount*) akun **Google Drive** ke Google Colab.

### Langkah-Langkah Integrasi Google Drive di Colab:

1. Hubungkan Google Drive dengan modul bawaan `google.colab`. 2. Berikan izin akses saat muncul jendela verifikasi akun Google. 3. Seluruh folder dan berkas di Google Drive Anda akan tersedia di jalur direktori `/content/drive/MyDrive/`.

PYTHON 3

Contoh Kode Praktik

```
# Skrip khusus lingkungan Google Colab:
# from google.colab import drive
# drive.mount('/content/drive')

# Jalur akses dataset yang tersimpan di Google Drive:
# path_file = '/content/drive/MyDrive/Proyek_Data_Science/dataset_transaksi.csv'
# df_drive = pd.read_csv(path_file)
# print(df_drive.head())

print("Sintaks Mount Google Drive di Google Colab:")
print("from google.colab import drive")
print("drive.mount('/content/drive')")
print("df = pd.read_csv('/content/drive/MyDrive/path_folder/dataset.csv')")
```

#### OUTPUT EKSEKUSI:

```
Sintaks Mount Google Drive di Google Colab:
from google.colab import drive
drive.mount('/content/drive')
df = pd.read_csv('/content/drive/MyDrive/path_folder/dataset.csv')
```

## 5. Membaca & Mengekspor Format Microsoft Excel (`.xlsx`)

Selain CSV, berkas spreadsheet Microsoft Excel (`.xlsx`) sangat banyak digunakan dalam laporan bisnis. Untuk membaca dan menulis format Excel, Pandas membutuhkan modul pihak ketiga bernama **openpyxl** (`pip install openpyxl`).

Gunakan fungsi `pd.read_excel()` untuk membaca dan method `df.to_excel()` untuk mengekspor.

```
# Instalasi modul pendukung Excel jika belum terpasang:
# ! pip install openpyxl

# Contoh ekspor DataFrame ke berkas Excel (.xlsx):
# df_produk.to_excel('laporan_inventaris.xlsx', sheet_name='Ringkasan_Stok', index=False)

# Contoh membaca sheet tertentu dari berkas Excel:
# df_excel = pd.read_excel('laporan_inventaris.xlsx', sheet_name='Ringkasan_Stok')

print("Fungsi Ekspor Excel: df.to_excel('laporan.xlsx', sheet_name='Sheet1', index=False)")
print("Fungsi Baca Excel : pd.read_excel('laporan.xlsx', sheet_name='Sheet1')")
```

**OUTPUT EKSEKUSI:**

```
Fungsi Ekspor Excel: df.to_excel('laporan.xlsx', sheet_name='Sheet1', index=False)
Fungsi Baca Excel : pd.read_excel('laporan.xlsx', sheet_name='Sheet1')
```

## 6. Membaca & Mengekspor Format JSON ( `pd.read_json` & `df.to_json` )

JSON (*JavaScript Object Notation*) adalah format transmisi standar untuk integrasi API dan aplikasi web modern.

```
# 1. Ekspor DataFrame ke berkas JSON (orient='records' menyusun data per baris objek)
df_produk.to_json('daftar_produk.json', orient='records', indent=2)
print("Berkas 'daftar_produk.json' berhasil diekspor!")

# 2. Membaca kembali dari berkas JSON
df_json = pd.read_json('daftar_produk.json')
print("\nHasil Baca Berkas JSON:\n", df_json)
```

**OUTPUT EKSEKUSI:**

```
Berkas 'daftar_produk.json' berhasil diekspor!
```

```
Hasil Baca Berkas JSON:
```

|   | Kode | Produk             | Harga    | Stok |
|---|------|--------------------|----------|------|
| 0 | A01  | Laptop Pro         | 14500000 | 12   |
| 1 | A02  | Mouse Wireless     | 250000   | 45   |
| 2 | A03  | Keyboard Mekanikal | 850000   | 28   |
| 3 | A04  | Monitor 4K         | 4200000  | 15   |

## 7. Rangkuman Tabel Fungsi Impor & Ekspor Data

| Sumber / Format | Fungsi Membaca (*Import*) | Fungsi Menyimpan (*Export*) | Kebutuhan Tambahan / Parameter Kunci |

| :--- | :--- | :--- | :--- |

|                                |                                      |                                                 |                       |
|--------------------------------|--------------------------------------|-------------------------------------------------|-----------------------|
| CSV Lokal                      | <code>pd.read_csv('file.csv')</code> | <code>df.to_csv('file.csv', index=False)</code> | <code>sep=';',</code> |
| <code>encoding='utf-8',</code> | <code>usecols</code>                 |                                                 |                       |

| **URL Web / GitHub** | `pd.read_csv('https:// ...')` | - | Mendukung protokol HTTP/HTTPS langsung |  
| **Git Clone** | `!git clone` lalu `pd.read_csv(...)` | `git push` via CLI | Menarik seluruh folder repositori data |  
| **Google Drive** | `from google.colab import drive` + `drive.mount('/content/drive')` |  
`df.to_csv('/content/drive/MyDrive/ ...')` | Khusus Google Colab untuk persistensi cloud |  
| **Excel (.xlsx)** | `pd.read_excel('file.xlsx', sheet_name=...)` | `df.to_excel('file.xlsx', index=False)` |  
Memerlukan pustaka `openpyxl` |  
| **JSON** | `pd.read_json('file.json')` | `df.to_json('file.json', orient='records')` | Format standar API web dan NoSQL |

## Tugas Praktik: Mini Proyek Analisis & Visualisasi Produk (Tantangan Mandiri)

### Instruksi Tantangan:

Tugas integrasi Bab 2 ini dirancang untuk **menguji daya nalar dan pemahaman mandiri** Anda setelah mempelajari ekosistem paket Python: **Pip, NumPy, Pandas, Matplotlib**, dan **Seaborn**.

 **Tantangan Anda:** Alih-alih hanya menjalankan kode yang sudah siap pakai, Anda diminta **memikirkan logika dan melengkapi bagian-bagian kode yang masih rumpang** ( \_\_\_\_\_ ).

Silakan klik tombol **Buka di Google Colab** atau unduh file `.ipynb` ini untuk mengisi dan menguji langsung program Anda.

### Tolok Ukur Hasil (Expected Output)

Sebelum mulai menulis kode, cermati target output teks dan grafik di bawah ini. Kode yang Anda lengkapi harus mampu menghasilkan komputasi dan visualisasi serupa:

```
# [Target Output Visualisasi & Komputasi]
# Perhatikan acuan diagram batang dan output konsol di bawah ini sebelum Anda melengkapi
soal-soal berikut.
```

#### OUTPUT EKSEKUSI:

```
=== TARGET OUTPUT SOAL 1 & 2 ===
      Kode      Produk      Harga  Stok  Total_Nilai
0  A01      Laptop Pro  14500000    12    174000000
1  A02      Mouse Wireless    250000    45    11250000
2  A03  Keyboard Mekanikal    850000    28    23800000
3  A04      Monitor 4K    4200000    15    63000000

=== TARGET RINGKASAN SAINTIFIK (NUMPY) ===
Total Nilai Aset Inventaris : Rp 272,050,000
Rata-rata Harga Produk      : Rp 4,950,000
Rata-rata Stok Unit         : 25.0 unit
```

#### OUTPUT EKSEKUSI:

```
<Figure size 960x540 with 1 Axes>
```

## Soal 1: Membaca Dataset dengan Pandas

**Instruksi:** Panggil fungsi Pandas yang tepat untuk membaca file `daftar_produk.csv` pada bagian kosong

```
-----
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# TODO 1: Ganti '_____' dengan nama method Pandas untuk membaca file CSV
df = pd._____('daftar_produk.csv')

print("Dataset Berhasil Dimuat:")
print(df)
print(f"\nDimensi Data: {df.shape[0]} baris, {df.shape[1]} kolom")
```

## Soal 2: Menghitung Kolom Vektor & Statistik Agregasi NumPy

**Instruksi:**

1. Lengkapi kalkulasi vektor untuk kolom baru `Total_Nilai` yang merupakan perkalian `Harga` dan `Stok`.
2. Gunakan fungsi statistik NumPy (`np.sum` dan `np.mean`) pada bagian `_____` untuk menghitung total aset dan rata-rata.

PYTHON 3

Contoh Kode Praktik

```
# TODO 2a: Lengkapi perkalian kolom vektor (Harga dikali Stok)
df['Total_Nilai'] = df['_____'] * df['_____']

# TODO 2b: Lengkapi fungsi statistik NumPy yang tepat (sum / mean)
total_aset = np._____(df['Total_Nilai'])
rata_harga = np._____(df['Harga'])
rata_stok  = np._____(df['Stok'])

print("Hasil Penambahan Kolom Baru (Total_Nilai):")
print(df[['Kode', 'Produk', 'Harga', 'Stok', 'Total_Nilai']])
print("\n=== Ringkasan Statistik Saintifik (NumPy) ===")
print(f"Total Nilai Aset Inventaris : Rp {total_aset:,.0f}")
print(f"Rata-rata Harga Produk      : Rp {rata_harga:,.0f}")
print(f"Rata-rata Stok Unit          : {rata_stok:.1f} unit")
```

---

### Soal 3: Visualisasi Diagram Batang (Bar Chart)

**Instruksi:** Masukkan kolom kategori produk untuk sumbu X dan total nilai stok untuk sumbu Y, lalu berikan label sumbu yang informatif pada bagian `_____`.

```
# Mengatur tema grafik modern
sns.set_theme(style='whitegrid')
fig, ax = plt.subplots(figsize=(8, 4.5))

# TODO 3a: Tentukan kolom sumbu X (kategori produk) dan sumbu Y (total nilai dalam jutaan)
palette = sns.color_palette("viridis", len(df))
bars = ax.bar(df['_____'], df['_____'] / 1_000_000, color=palette, edgecolor='black',
linewidth=0.8)

# TODO 3b: Berikan label nama sumbu yang deskriptif
ax.set_title("Nilai Total Inventaris per Produk (Juta Rupiah)", fontsize=13,
fontweight='bold', pad=12)
ax.set_xlabel("_____", fontsize=10, fontweight='bold')
ax.set_ylabel("_____", fontsize=10, fontweight='bold')
ax.grid(axis='y', linestyle='--', alpha=0.7)

# Menambahkan label nilai di atas batang
for bar in bars:
    yval = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2.0, yval + 2, f"{yval:.1f} jt", ha='center',
va='bottom', fontsize=9, fontweight='bold')

plt.tight_layout()
plt.show()
```

#### Soal 4: Ekspor DataFrame ke File CSV Bersih

**Instruksi:** Lengkapi method ekspor DataFrame ke file `laporan_inventaris_selesai.csv` tanpa mengikutsertakan indeks bawaan (`index=False`).

```
# TODO 4: Lengkapi fungsi ekspor file CSV dan parameter index
output_file = 'laporan_inventaris_selesai.csv'
df._____(output_file, index=_____)

print(f"Laporan berhasil diekspor ke: {output_file}")
print("Selamat, Anda telah berhasil menyelesaikan seluruh tantangan mandiri Bab 2!")
```